Design Patterns- Creational

Ganesha

Content

1 What are design Patterns
- Definition
- Characteristics
- Benefits
- Categorizing

2 What are Creational Design Patterns
- Singleton Pattern
- Prototype Pattern
- Build Pattern
- Factory Pattern
- Abstract Factory Pattern

What are Design Patterns

Dictionary meaning of Pattern: "A plan, diagram, or model to be followed in making things"

A software designer or an architect needs to understand more than the relevant APIs, which includes

- What are the best practices?
- What are the bad practices?
- What are the common recurring problems and proven solutions to these problems?
- How is code re-factored from a less optimal scenario, or bad practice, to a better one typically described by a pattern?

What are Design Patterns

Some Definitions of Design Pattern

A design pattern is like a template that can be applied in many different situations.

Patterns are neither code, nor designs as they must be instantiated and applied.

- evaluate trade-offs and impact of using a pattern in the system at hand
- make design and implementation decision how best to apply the pattern, perhaps modify it slightly
- implement the pattern in code and combine it with other patterns

"A design pattern is a general repeatable solution to a commonly occurring problem in software design. It is a description or template for how to solve a problem that can be used in many different situations."

What are Design Patterns

There are many definitions for a pattern, but all these definitions have a common theme. Some of the common characteristics of patterns are:

- Patterns are observed through experience.
- Patterns are typically written in a structured format called Template.
- Patterns prevent reinventing the wheel.
- Patterns exist at different levels of abstraction.
- Patterns undergo continuous improvement.
- Patterns are reusable artifacts.
- Patterns can be used together to solve a larger problem.

Design Patterns - Benefit

Benefits of Using Patterns:

- Patterns are a common design vocabulary
  - they allow engineers to abstract a problem and talk about that abstraction in isolation from its implementation.
- Patterns capture design expertise and allow that expertise to be communicated
- Promote design reuse and avoid mistakes
- Improve documentation (less is needed) and understandability (patterns are described well once)

Design Patterns - Categorizing

Patterns have been captured at many levels of abstraction and in numerous domains. Numerous categories have been suggested for classifying software patterns, with some of the most common being

- creational patterns

abstract the object instantiation process

- structural patterns

describe how classes and objects can be combined to form larger Structures

- behavioral patterns

concerned with communication, algorithms and the assignment of responsibilities between objects

Creational Patterns

This Design pattern is all about class instantiation. Defines the best possible way in which an object can be instantiated

- Make system independent of how objects are created composed and represented

- Class-creation:
  - use inheritance effectively in the instantiation process

- Object-creation:
  - use delegation effectively to get the job done

Creational Patterns

- Factory Method Pattern

Define an interface for creating an object, but let subclasses decide which class to instantiate.

- Abstract Factory Pattern

Provide an interface for creating families of related or dependent objects without specifying their concrete classes.

- Builder Pattern

Separate the construction of a complex object from its representation so that the same construction process can create different representations

- Prototype Pattern

Specify the kinds of objects to create using a prototypical instance, and create new objects by copying this prototype.

- Singleton Pattern

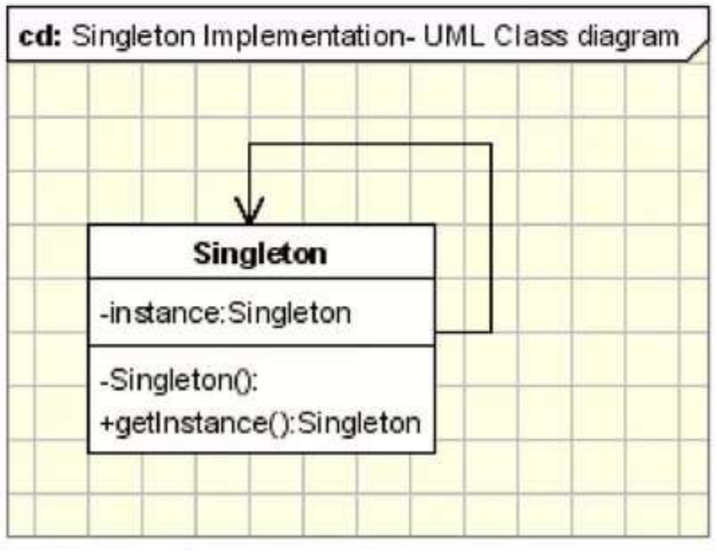
Ensure a class has only one instance and provide a global point of access to it.

Singleton

Motivation:
Sometimes it's important to have only one instance for a class. For example, in a system there should be only one window manager (or only a file system or only a print spooler).

Intent
- Ensure that only one instance of a class is created.
- Provide a global point of access to the object

Implementation
The implementation involves a static member in the "Singleton" class, a private constructor and a static public method that returns a reference to the static member.



Singleton

Typically the class code looks like

```
class Singleton {
        private static Singleton instance;
        private static lockObject = new Object()
        private Singleton()      {
                ...
        }
        public static Singleton getInstance() {
                if (instance == null)
                {
                    lock(lockObject)
                    {
                        if (instance == null)
                            instance = new Singleton();
                    }
                }
                return instance;
        }
        public void doSomething() {
                ...
        }
}
```

Applicability & Examples

- Logger Classes
- Configuration Classes
- Accessing resources in shared mode
- Factories implemented as Singletons

Prototype

Motivation:

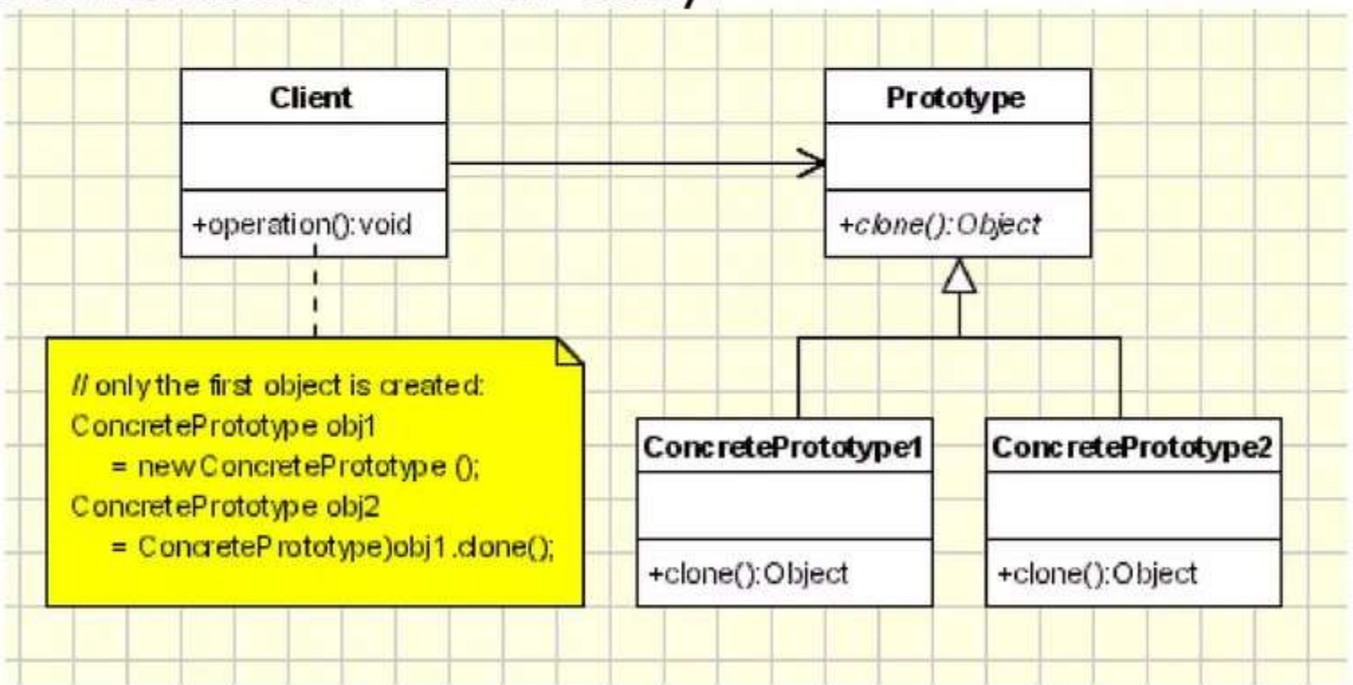
- If the cost of creating a new object is large and creation is resource intensive, we clone the object.
- We clone the existing object and change the state of the object as needed.

Intent:

- specifying the kind of objects to create using a prototypical instance
- creating new objects by copying this prototype object to avoid creation

Implementation

The pattern uses abstract classes, and only three types of classes make its implementation rather easy.



- **Client** - creates a new object by asking a prototype to clone itself.
- **Prototype** - declares an interface for cloning itself.
- **ConcretePrototype** - implements the operation for cloning itself.

Prototype

Typically the class code looks like

```
public interface Prototype
{
        public  abstract public interface Prototype Clone( );
}

public class ConcretePrototype  : Prototype
{
        public override Prototype Clone()
        {
                //Shallow copy
            return (Prototype)this.MemberwiseClone();
        }
}

public class Client
 {

        public static void main( String arg[] )
        {
                ConcretePrototype obj1= new ConcretePrototype ();
                ConcretePrototype obj2 = (ConcretePrototype)obj1.Clone();
        }
}
```

Prototype

Applicability

Use Prototype Pattern when a system should be independent of how its products are created, composed, and represented.

- Classes to be instantiated are specified at run-time for example, by dynamic loading
- It is more convenient to copy an existing instance than to create a new one.
- When instances of a class can have one of only a few different combinations of state.

Examples

- Analysis on a set of data from the database.

Factory

Motivation:
Helps to model an interface for creating an object , which at creation time can let its subclasses decide which class to instantiate.

Intent:

- creates objects without exposing the instantiation logic to the client.
- refers to the newly created object through a common interface.
- In programmer's language (very raw form), you can use factory pattern where you have to create an object of any one of sub-classes depending on the data provided.

Advantages:

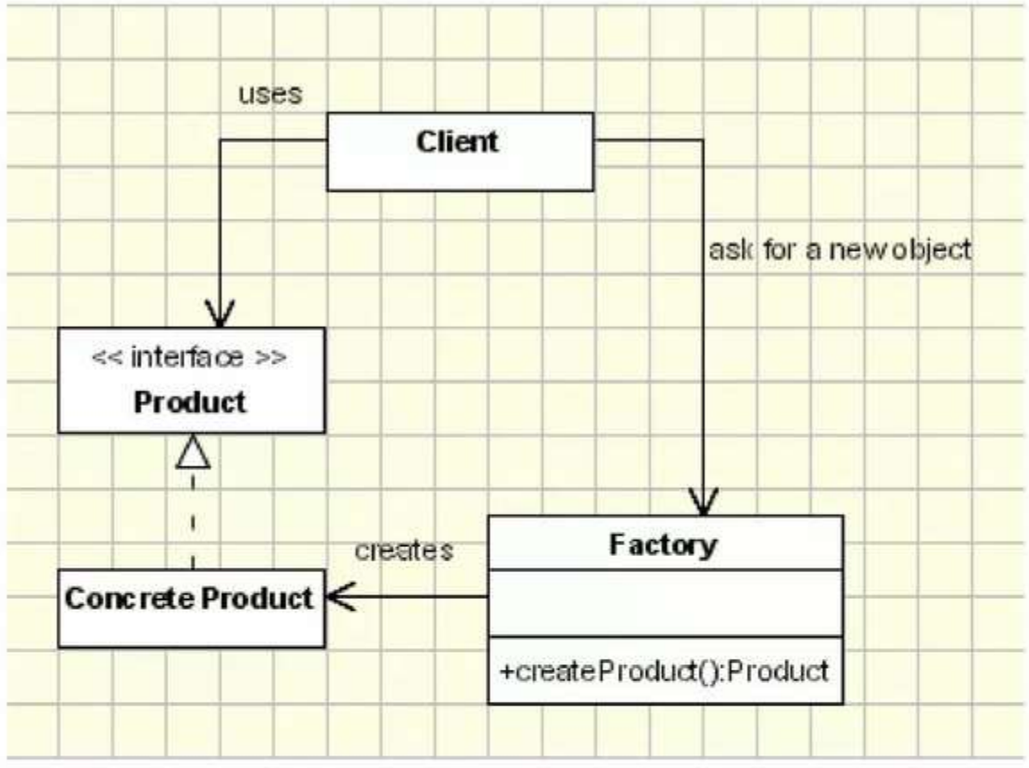
- The use of factories gives the programmer the opportunity to abstract the specific attributes of an Object into specific subclasses which create them.
- The Factory Pattern promotes loose coupling by eliminating the need to bind application-specific classes into the code.

Factory

Implementation

Simplest Implementation

```
public Product createProduct(String ProductID){
                if (id==ID1)
                return new OneProduct();
                if (id==ID2) return
                return new AnotherProduct();
                ... // so on for the other Ids
                return null; //if the id doesn't have any
                of  the expected values
}
```



- The client needs a product, but instead of creating it directly using the new operator, it asks the factory object for a new product, providing the information about the type of object it needs.
- The **factory instantiates** a new concrete product and then returns to the client the newly created product(casted to abstract product class).
- The client uses the products as abstract products without being aware about their concrete implementation.

Abstract Factory

Motivation:

- This pattern is one level of abstraction higher than factory pattern.
- Abstract Factory is a super-factory which creates other factories
- Provide an interface for creating families of related or dependent objects.
- Factory method takes care of one product where as the abstract factory Pattern provides a way to encapsulate a family of products.

Intent:

- Provide an interface for creating families of related or dependent objects without explicitly specifying their classes.

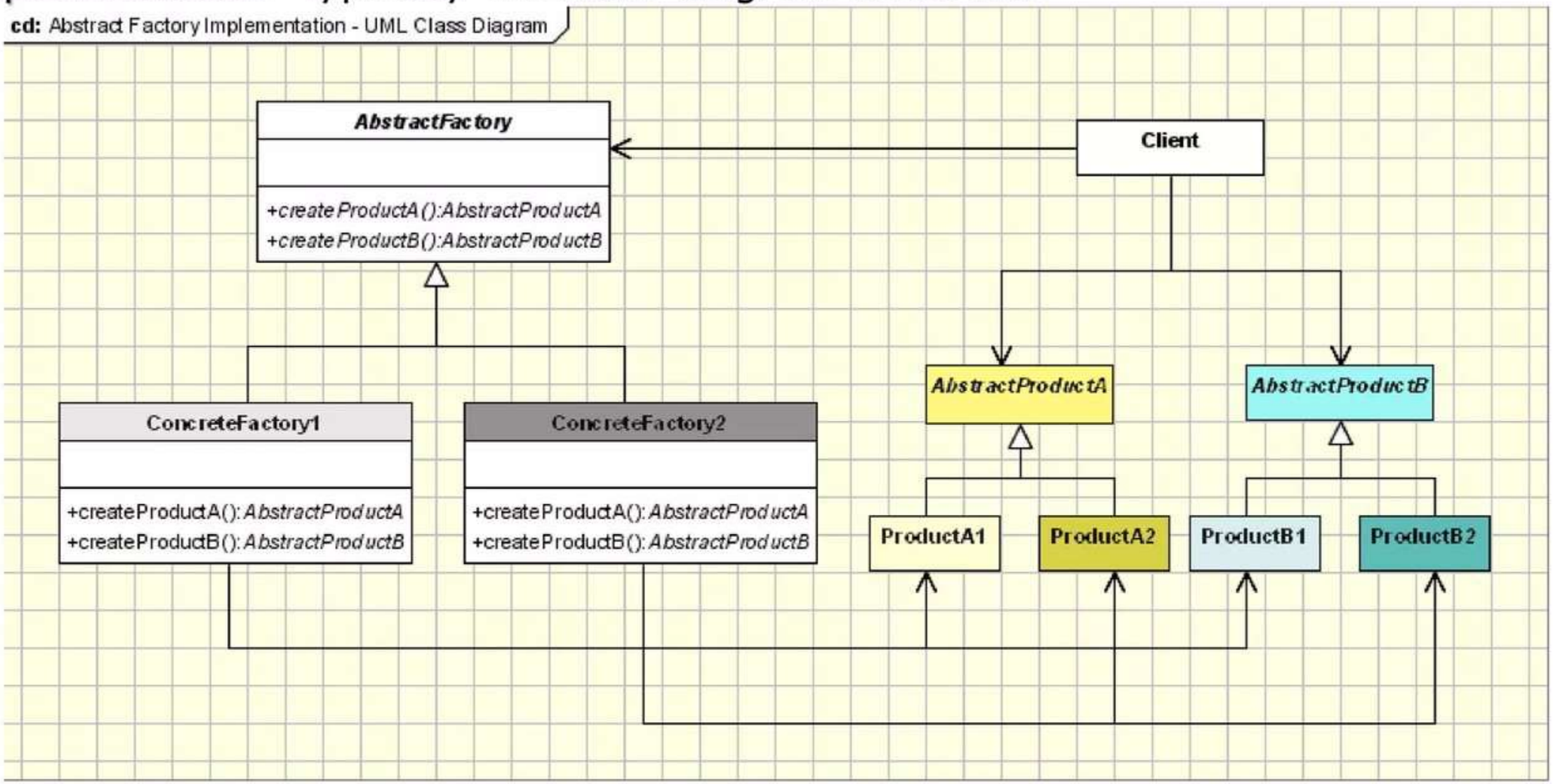
Applicability:

- A system should be independent of how its products are created, composed, and represented.

- If you want to provide a class library products, and you want to reveal just their interfaces , not their implementations.

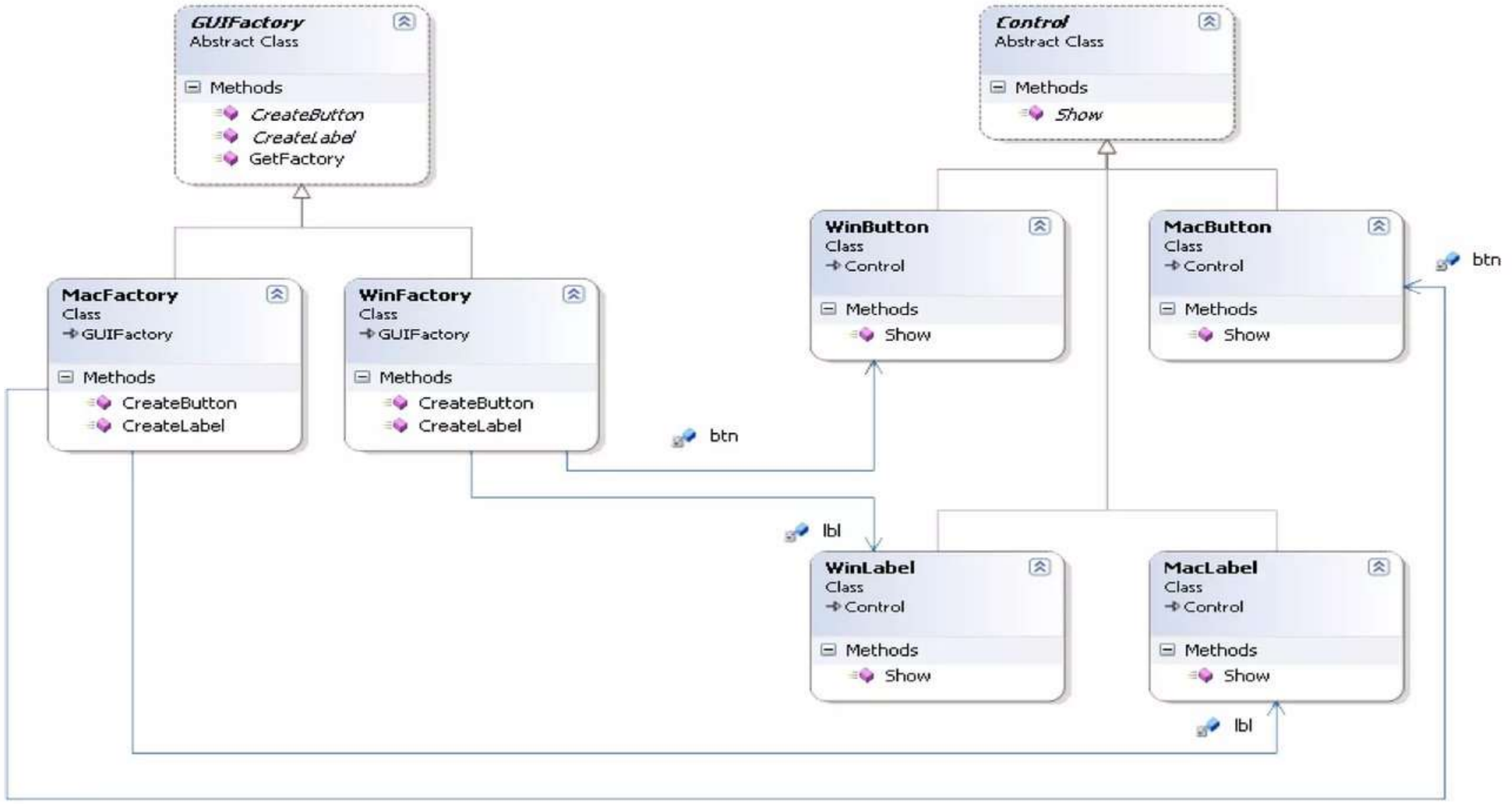
Abstract Factory

Implementation Typically the class diagram looks like



Abstract Factory

Example: We have a requirement where we need to create control library and the same library supports multiple platforms but the client code should not be changed if we import from one operating system to the other. The solution is



Builder

Motivation:
Builder, as the name suggests builds complex objects from simple ones step-by-step. It separates the construction of complex objects from their representation.

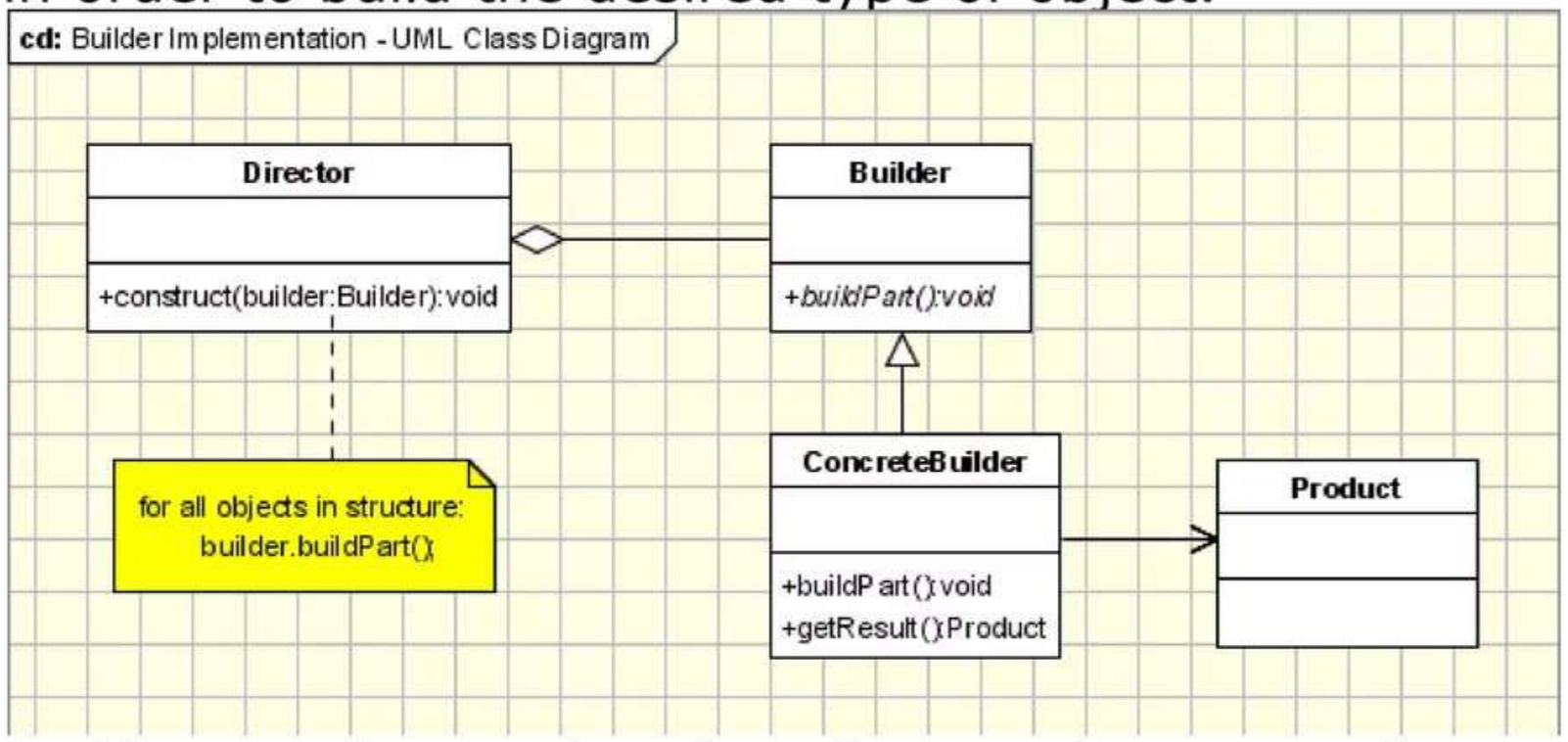
Intent:
- Defines an instance for creating an object but letting subclasses decide which class to instantiate using any of the patterns.
- Refers to the newly created object through a common interface

Advantages:
- Encapsulation: The builder pattern encapsulates the construction process of a complex object and thereby increases the modularity of an application.

Builder

Implementation

The Builder design pattern uses the Factory Builder pattern to decide which concrete class to initiate in order to build the desired type of object.



- The **Builder** class specifies an abstract interface for creating parts of a Product object.
- The **ConcreteBuilder** constructs and puts together parts of the product by implementing the Builder interface. It defines and keeps track of the representation it creates and provides an interface for saving the product.
- The **Director** class constructs the complex object using the Builder interface.
- The **Product** represents the complex object that is being built..

Builder

Applicability:

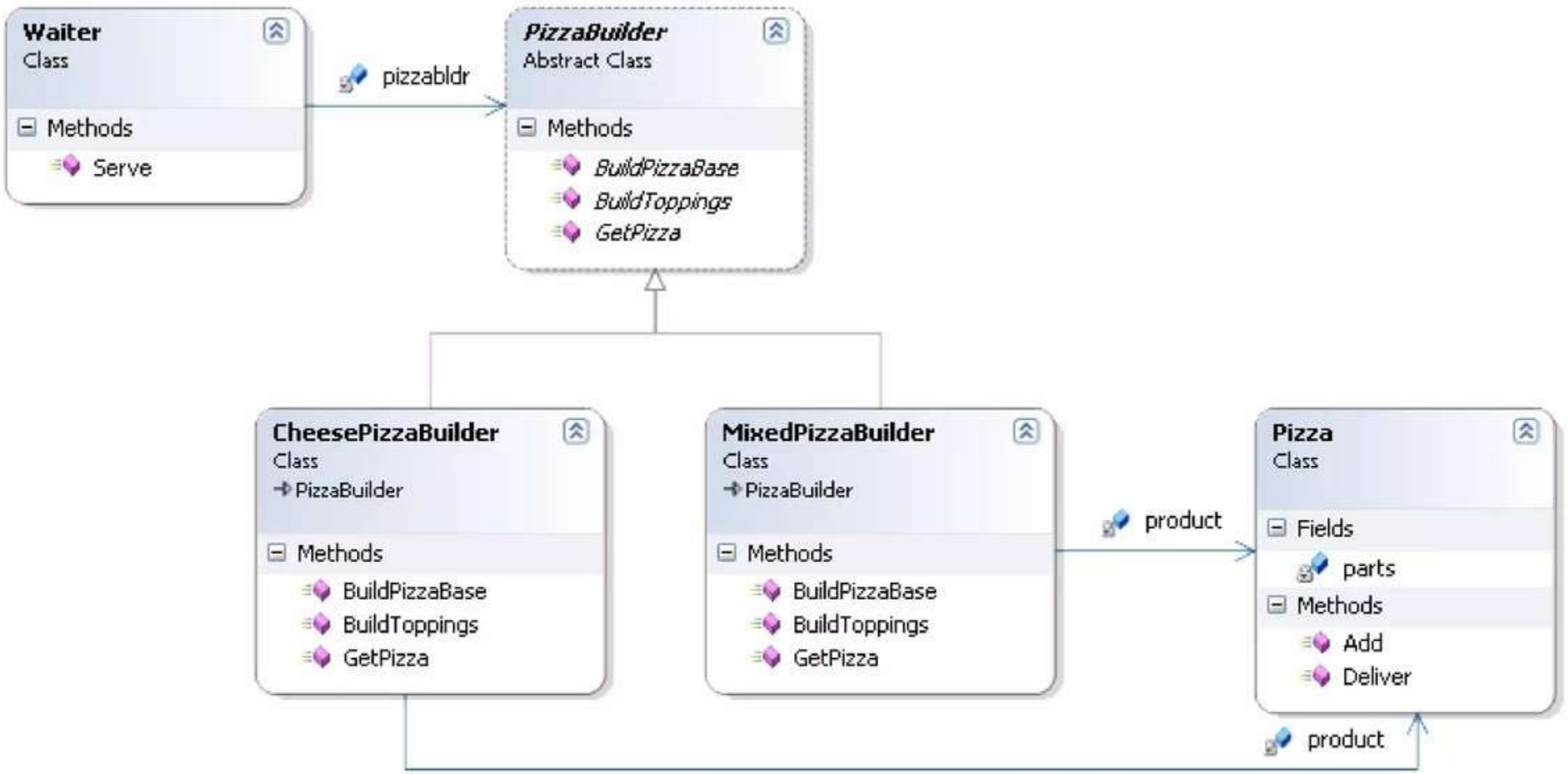
- Builder patterns is useful in a large system
- Builder patterns fits when different variations of an object may need to be created and the inheritance into those objects is well-defined.
- the creation algorithm of a complex object is independent from the parts that actually compose the object

Example:

- User specific UI:
  the admin needs to have some buttons enabled, buttons that needs to be disabled for the common user.

The **Builder** provides the interface for building form depending on the login information.
The **ConcreteBuilders** are the specific forms for each type of user.
The **Product** is the final form that the application will use in the given case
The **Director** is the application that, based on the login information, needs a specific form.

Builder


Waiter
Class
Methods
Serve
pizzabldr
PizzaBuilder
Abstract Class
Methods
BuildPizzaBase
BuildToppings
GetPizza
CheesePizzaBuilder
Class
PizzaBuilder
Methods
BuildPizzaBase
BuildToppings
GetPizza
MixedPizzaBuilder
Class
PizzaBuilder
Methods
BuildPizzaBase
BuildToppings
GetPizza
product
Pizza
Class
Fields
parts
Methods
Add
Deliver
product

Comparison

	Abstract Factory	Factory Method	Builder	Prototype	Singleton
When to use	Need complete flexibility - new kinds of derived classes could be created later which would be usable by the client	Need limited flexibility - choose from among a limited set of configurations	Don't want the details, want a configuration which will enable me to start working on it right away	Don't want the details, don't even want to pick what package I will get - it will probably be passed to me	Want to create only one of these - could be known to me, or could be passed to me
Client knowledge	Less knowledge of type - client knows base type, not specific class. Knowledge of class flexible, but knowledge of elements of the configuration, it has to create them explicitly	More knowledge - client knows specific type, and knows about the internal configuration - it has to create them explicitly	Less knowledge of details, no knowledge of configuration	More knowledge of details, some knowledge of configuration - enough to make the right choice	Orthogonal to how much client has to know, but knowledge limited by the fact that most of the time, it uses an instance that already exists
Advantages/ Disadvantages	Great flexibility, Low client knowledge	Reduced flexibility, High client knowledge	Reduced flexibility, minimal client knowledge	Good flexibility, minimal client knowledge	Variable flexibility, minimal client knowledge

Questions and answers

Thank You